

WAFt Auto-Work

Autonomous Work Effort Execution Guide

A comprehensive guide to the WAFt Auto-Work feature

WAFt System | AI Assistant

January 2026

The WAFT Auto-Work feature represents a significant advancement in autonomous project management. This system enables intelligent, hands-off execution of work efforts by analyzing priorities, selecting optimal tasks, and executing them autonomously with comprehensive safety mechanisms.

0.1. Purpose of This Guide

This guide provides:

- **[Comprehensive Documentation]**: Complete explanation of how Auto-Work functions
- **[Step-by-Step Walkthrough]**: Detailed process flow from start to finish
- **[Usage Examples]**: Real-world scenarios with expected outputs
- **[Integration Details]**: How Auto-Work integrates with Empirica, Pantheon, Campfire, and D&D systems
- **[Safety Mechanisms]**: Security features and validation processes
- **[Troubleshooting]**: Common issues and solutions

0.2. Who This Guide Is For

This guide is designed for:

- **[Developers]**: Understanding the technical implementation
- **[Project Managers]**: Learning how to leverage autonomous work execution
- **[System Administrators]**: Configuring and maintaining the system
- **[Users]**: Learning to use Auto-Work effectively

0.3. What You'll Learn

By the end of this guide, you will understand:

1. How Auto-Work analyzes and prioritizes work efforts
2. The priority scoring algorithm and its factors
3. How safety gates prevent unsafe operations
4. Integration with epistemic tracking (Empirica)
5. Integration with decision support (Pantheon)
6. Storytelling integration (Campfire)
7. D&D campaign integration for quest generation
8. How to use Auto-Work effectively in your workflow

0.4. Prerequisites

Before using Auto-Work, ensure you have:

- WAFT installed and configured
- Work efforts in `_work_efforts/` directory
- (Optional) Empirica initialized for epistemic tracking
- (Optional) Pantheon entities configured for decision support
- (Optional) Campfire configured for storytelling
- (Optional) D&D campaign system configured for quest generation

0.5. Document Structure

This guide is organized into five parts:

[Part I: Introduction & Overview] - What Auto-Work is and its key features

[Part II: How It Works] - Deep dive into the architecture and algorithms

[Part III: Integration & Safety] - Empirica, Pantheon, Campfire, D&D, and safety mechanisms

[Part IV: Usage Guide] - How to use Auto-Work with examples and walkthroughs

[Part V: Advanced Topics] - Customization, best practices, and future enhancements

0.6. Getting Started

To get started with Auto-Work, run:

```
/auto-work
```

Or to see what would be done without executing:

```
/auto-work --dry-run
```

For detailed output:

```
/auto-work --verbose
```

Let's begin by understanding what Auto-Work is and how it can transform your workflow.

Auto-Work is an autonomous work effort execution system that intelligently analyzes all available work efforts, calculates priorities, selects the best one to work on, determines the optimal action, and executes it autonomously.

0.7. Core Concept

Auto-Work embodies the principle of **[autonomous decision-making]**: instead of manually selecting which work effort to tackle next, the system uses sophisticated algorithms to make that decision for you.

0.8. The Problem It Solves

In complex projects with many work efforts, deciding what to work on next can be challenging:

- **[Too Many Options]**: Dozens or hundreds of work efforts to choose from
- **[Priority Confusion]**: Unclear which work effort is most important
- **[Context Switching]**: Time lost deciding between tasks
- **[Inconsistent Prioritization]**: Different criteria applied at different times
- **[Missed Opportunities]**: Important work efforts overlooked

Auto-Work solves these problems by:

1. **[Systematic Analysis]**: Evaluates all work efforts using consistent criteria
2. **[Intelligent Prioritization]**: Uses multi-factor scoring to rank work efforts
3. **[Autonomous Selection]**: Picks the best work effort automatically
4. **[Action Determination]**: Identifies the optimal action to take
5. **[Safe Execution]**: Executes with comprehensive safety checks

0.9. Key Characteristics

0.9.1. Autonomous

Auto-Work operates independently, making decisions without human intervention. It analyzes, selects, and prepares execution instructions autonomously.

0.9.2. Intelligent

The system uses sophisticated algorithms considering:

- Work effort status (active, paused, open, completed)
- Priority levels (CRITICAL, HIGH, MEDIUM, LOW)
- Content indicators (TODOs, FIXMEs, bugs)
- Recent activity (git commits)
- Epistemic state (Empirica)
- Precedent analysis (Pantheon)
- Repository context (GitHubGod)

0.9.3. Safe

Multiple safety mechanisms ensure safe operation:

- Empirica safety gates (PROCEED/HALT/BRANCH/REVISE)
- Pantheon Judge evaluation
- Action type whitelisting
- Work effort ID validation
- Command sanitization
- Execution logging

0.9.4. Integrated

Auto-Work integrates seamlessly with:

- **[Empirica]**: Epistemic tracking and safety gates

- **[Pantheon]**: Decision support entities (Judge, Magistrate, TheReasoner, GitHubGod, Librarian, MissionControl, Fae)
- **[Campfire]**: Storytelling around the campfire
- **[D&D Campaign]**: Quest PDF generation from scenarios

0.10. How It Differs from Manual Work

0.10.1. Manual Work Flow

1. Review all work efforts
2. Manually assess priorities
3. Select a work effort
4. Determine what action to take
5. Execute the action

0.10.2. Auto-Work Flow

1. System analyzes all work efforts
2. Calculates priority scores automatically
3. Selects best work effort automatically
4. Determines optimal action automatically
5. Prepares execution instruction
6. AI executes the action

0.11. Use Cases

Auto-Work is ideal for:

- **[Hands-Off Execution]**: Let the system work autonomously
- **[Intelligent Prioritization]**: Need consistent, multi-factor prioritization
- **[Large Projects]**: Many work efforts to manage
- **[Continuous Progress]**: Keep work moving forward automatically
- **[Learning Systems]**: Systems that learn from execution patterns

0.12. What Auto-Work Does NOT Do

Auto-Work does not:

- Execute destructive operations without safety checks
- Bypass security validations
- Work on completed work efforts
- Execute actions outside the whitelist
- Ignore Empirica safety gates

0.13. Next Steps

Now that you understand what Auto-Work is, let's explore its key features in detail.

Auto-Work includes several key features that make it powerful and reliable.

0.14. Intelligent Prioritization

Auto-Work uses a sophisticated multi-factor scoring algorithm to prioritize work efforts:

- Status weighting (active > paused > open)
- Priority levels (CRITICAL > HIGH > MEDIUM > LOW)
- Content indicators (TODOs, FIXMEs, bugs)
- Recent activity (git commits)
- Epistemic state (Empirica)
- Precedent analysis (Pantheon)

0.15. Autonomous Selection

The system automatically selects the best work effort without human intervention:

- Analyzes all available work efforts
- Calculates priority scores
- Selects highest scoring work effort
- Determines optimal action
- Prepares execution instruction

0.16. Safety Mechanisms

Multiple layers of security ensure safe operation:

- Input validation (work effort IDs, paths)
- Action whitelisting (only approved actions)
- Command sanitization (validate commands)
- Empirica safety gates (PROCEED/HALT/BANCH/REVISE)
- Pantheon Judge evaluation
- Execution logging

0.17. Integration Support

Auto-Work integrates with multiple WAFT systems:

- **[Empirica]**: Epistemic tracking and safety gates
- **[Pantheon]**: Decision support entities
- **[Campfire]**: Storytelling around the campfire
- **[D&D Campaign]**: Quest PDF generation

0.18. Flexible Execution

Multiple execution modes:

- **[Normal]**: Execute immediately
- **[Dry Run]**: Preview without executing
- **[Verbose]**: Detailed logging output

0.19. Graceful Degradation

System continues to work even if optional components are unavailable:

- Empirica unavailable: Continues without epistemic tracking
- Pantheon unavailable: Continues without decision support
- Campfire unavailable: Continues without storytelling
- D&D unavailable: Continues without quest generation

0.20. Comprehensive Logging

All operations are logged for audit and debugging:

- Work effort selection
- Priority calculations
- Safety gate results
- Execution outcomes
- Story generation
- Quest creation

0.21. Next Steps

Now that you understand the key features, let's explore the system architecture.

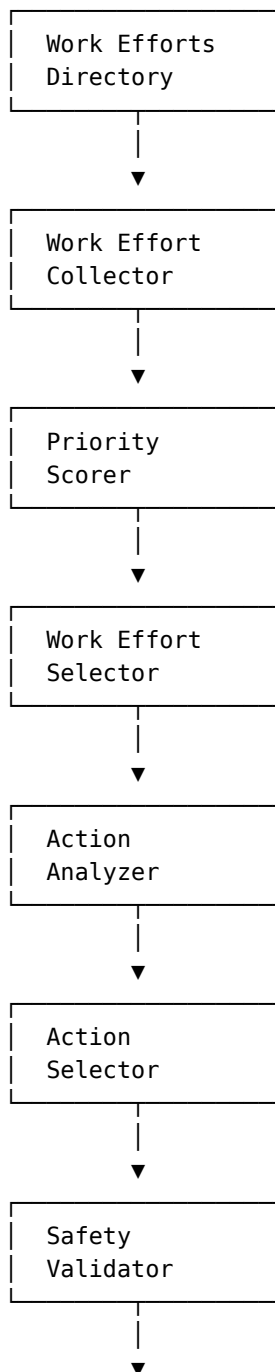
This chapter describes the architecture of the Auto-Work system.

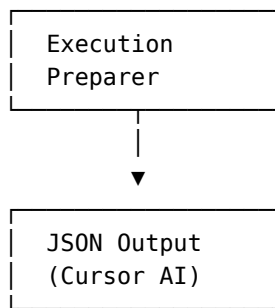
0.22. Component Overview

Auto-Work consists of several components:

1. **[Work Effort Collector]**: Gathers all work efforts
2. **[Priority Scorer]**: Calculates priority scores
3. **[Work Effort Selector]**: Selects best work effort
4. **[Action Analyzer]**: Analyzes available actions
5. **[Action Selector]**: Selects optimal action
6. **[Safety Validator]**: Validates safety
7. **[Execution Preparer]**: Prepares execution instruction
8. **[Integration Manager]**: Manages integrations

0.23. Data Flow





0.24. Integration Points

0.24.1. Empirica Integration

- Epistemic state assessment
- Safety gate checks
- Finding logging
- Uncertainty tracking

0.24.2. Pantheon Integration

- Judge: Action safety evaluation
- Magistrate: Precedent search
- TheReasoner: Reasoning traces
- GitHubGod: Repository state
- Librarian: Knowledge base search
- MissionControl: Mission coordination

0.24.3. Campfire Integration

- Story generation
- Narrative creation
- PDF output

0.24.4. D&D Campaign Integration

- Scenario execution
- Quest markdown generation
- Quest PDF generation

0.25. File Structure

```
scripts/
  auto_work.py          # Main script
  show_me.py            # Work effort collection
  work_dashboard.py     # Action analysis

src/waft/
  core/
    empirica.py         # Empirica integration
    science/
      oracle.py         # Oracle integration
    pantheon.py          # Pantheon entities
  core/
    campfire.py         # Campfire integration
  dnd_scenario/         # D&D campaign
```

0.26. Next Steps

Now that you understand the architecture, let's dive into the priority scoring algorithm.

The priority scoring algorithm is the heart of Auto-Work's decision-making. It calculates a numerical score for each work effort, with higher scores indicating higher priority.

0.27. Scoring Formula Overview

The total priority score is calculated as:

Total Score = Status Weight + Priority Level + Content Indicators +
Recent Activity + Empirica Adjustment + Pantheon Adjustment

0.28. Factor 1: Status Weighting

Work effort status is the primary factor, as active work should be prioritized:

Status	Points	Description
active	100.0	Currently in progress (highest priority)
paused	50.0	Temporarily stopped
open	30.0	Not yet started
completed	0.0	Finished (excluded from selection)

[Example]: An active work effort starts with 100 points, while a paused one starts with 50.

0.29. Factor 2: Priority Level

Explicit priority levels add to the base score:

Priority	Points	Description
CRITICAL	+50.0	Urgent, must be addressed
HIGH	+30.0	Important, should be done soon
MEDIUM	+15.0	Normal priority
LOW	+5.0	Can wait

[Example]: A CRITICAL work effort gets +50 points, while MEDIUM gets +15.

0.30. Factor 3: Content Indicators

The system analyzes work effort content for indicators of work needed:

Indicator	Points	Description
Contains TODO	+20.0	Tasks to be done
Contains FIXME	+25.0	Issues to be fixed
Contains bug/error	+15.0	Bugs or errors mentioned

[Example]: A work effort with both TODO and FIXME gets +45 points.

0.31. Factor 4: Recent Activity

Recent git commits indicate active work:

- +5.0 points per commit in last 7 days
- Maximum: 20.0 points (capped at 4 commits)

[Example]: A work effort with 3 commits in the last week gets +15 points.

0.32. Factor 5: Empirica Adjustment

When Empirica is available, the system uses epistemic gates to adjust priority:

Gate Result	Adjustment	Description
PROCEED	+10.0	High confidence, ready to proceed
HALT	+20.0	Requires attention, boost priority
BRANCH	+15.0	Needs investigation, moderate boost
REVISE	0.0	No adjustment
None	0.0	Empirica unavailable

[Example]: If Empirica gate returns HALT, the work effort gets +20 points.

0.33. Factor 6: Pantheon Adjustments

Pantheon entities provide additional context:

0.33.1. Judge Evaluation

The Judge evaluates work effort readiness:

- PROVEN with confidence > 0.7: +15.0 points

- DISPROVEN with confidence > 0.7: -10.0 points
- PROBABLE with confidence > 0.6: +8.0 points

0.33.2. Magistrate Precedents

Similar work efforts found:

- +5.0 points per proven precedent (max 3 precedents)

0.33.3. Librarian Knowledge Base

Work effort referenced in knowledge base:

- +3.0 points per related record (max 3 records)

0.33.4. GitHubGod Repository State

Work effort branch matches current branch:

- +10.0 points if branch matches

0.34. Complete Example Calculation

Let's calculate the score for a work effort:

[Work Effort Details]:

- Status: active → 100.0 points
- Priority: HIGH → +30.0 points
- Contains TODO → +20.0 points
- Contains FIXME → +25.0 points
- 2 commits in last 7 days → +10.0 points
- Empirica gate: PROCEED → +10.0 points
- Judge: PROVEN (confidence 0.8) → +15.0 points
- Magistrate: 2 proven precedents → +10.0 points
- GitHubGod: Branch matches → +10.0 points

[Total Score]: 100 + 30 + 20 + 25 + 10 + 10 + 15 + 10 + 10 = **[230.0 points]**

0.35. Score Comparison

Work efforts are sorted by total score, with the highest scoring work effort selected.

0.36. Score Visualization

Work Effort A:  230.0 (Selected)
 Work Effort B:  150.0
 Work Effort C:  100.0
 Work Effort D:  50.0

0.37. Important Notes

- Completed work efforts always score 0.0 and are excluded
- Scores are calculated dynamically on each run
- Empirica and Pantheon adjustments are optional (graceful degradation)
- Content analysis is case-insensitive
- Git activity is limited to the last 7 days

0.38. Next Steps

Now that you understand priority scoring, let's see how work efforts are selected.

This chapter describes how Auto-Work selects the best work effort.

0.39. Selection Process

1. Collect all work efforts
2. Filter out completed work efforts
3. Calculate priority scores
4. Sort by score (highest first)
5. Select highest scoring work effort

0.40. Filtering

Completed work efforts are excluded:

- Status: completed → Score: 0.0 → Excluded

0.41. Sorting

Work efforts are sorted by total priority score:

Highest Score → Selected

Second Highest → Not selected

Third Highest → Not selected

...

0.42. Selection Example

Work Effort Scores:

WE-260118-abc1: 230.0 points  Selected

WE-260118-def2: 150.0 points

WE-260117-ghi3: 100.0 points

WE-260116-jkl4: 50.0 points

0.43. Next Steps

Now let's explore action determination.

This chapter describes how Auto-Work determines the optimal action.

0.44. Available Actions

Common action types:

- `status_transition`: Change work effort status
- `add_progress`: Add progress note
- `review`: Review work effort
- `review_todos`: Review and address TODOs
- `fix_issues`: Fix issues
- `review_changes`: Review recent changes

0.45. Action Priority

Actions are prioritized:

- `high` → Highest priority
- `medium` → Medium priority
- `low` → Lowest priority

0.46. Selection Process

1. Analyze available actions for selected work effort
2. Sort by priority (`high` > `medium` > `low`)
3. Select highest priority action

0.47. Example

Available Actions:

1. Address TODOs (HIGH priority) ☒ Selected
2. Review changes (MEDIUM priority)
3. Add progress note (LOW priority)

0.48. Next Steps

Now let's explore the execution phase.

This chapter describes the execution phase of Auto-Work.

0.49. Execution Steps

1. Validate action safety
2. Run Empirica gate check
3. Run Pantheon Judge evaluation
4. Prepare execution instruction
5. Output JSON for Cursor AI
6. (Optional) Generate story
7. (Optional) Generate quest PDF

0.50. Safety Validation

Before execution:

- Validate work effort ID format
- Check action type whitelist
- Sanitize command
- Run Empirica gate
- Run Judge evaluation

0.51. JSON Output

Structured JSON output for Cursor AI:

```
{
  "selected_work_effort": {
    "id": "WE-260118-abc1",
    "title": "Implement User Authentication",
    "path": "_work_efforts/WE-260118-abc1_implement_user_authentication"
  },
  "action": {
    "type": "review_todos",
    "label": "Address TODOs",
    "command": "Review and address TODOs in work effort WE-260118-abc1",
    "context": {
      "reason": "Work effort contains TODO items",
      "priority": "high"
    }
  },
  "execution_instruction": "Review and address TODOs in work effort WE-260118-abc1"
}
```

0.52. AI Execution

Cursor AI reads the JSON and executes the command autonomously.

0.53. Next Steps

Now let's explore Empirica integration.

Auto-Work integrates with Empirica for epistemic tracking and safety gates.

0.54. What is Empirica?

Empirica is an epistemic self-assessment system that tracks:

- Knowledge state (what you know)
- Uncertainty levels
- Engagement metrics
- Execution state
- Learning progress

0.55. Integration Points

0.55.1. Priority Adjustment

Empirica gates inform priority scoring:

- PROCEED: +10.0 points (high confidence)
- HALT: +20.0 points (requires attention)
- BRANCH: +15.0 points (needs investigation)
- REVISE: 0.0 points (no adjustment)

0.55.2. Safety Gates

Empirica provides safety gates before execution:

Gate Check → PROCEED/HALT/BRANCH/REVISE → Action

0.55.3. Finding Logging

Auto-Work logs findings to Empirica:

- Work effort selection
- Action execution
- Safety gate results
- Story generation
- Quest creation

0.56. Example Integration

 Empirica: Active and monitoring

 Empirica Gate: Checking execution safety...
Result: PROCEED

 Execution approved by Empirica gate

0.57. Next Steps

Now let's explore Pantheon integration.

Auto-Work integrates with Pantheon entities for decision support.

0.58. Pantheon Entities

0.58.1. Judge

Evaluates action safety:

- Verdict: PROVEN/DISPROVEN/PROBABLE
- Confidence: 0.0-1.0
- Reasoning: Explanation

0.58.2. Magistrate

Searches for precedents:

- Similar work efforts
- Proven patterns
- Historical outcomes

0.58.3. TheReasoner

Creates reasoning traces:

- Decision reasoning
- Context capture
- Outcome tracking

0.58.4. GitHubGod

Provides repository context:

- Current branch
- Repository state
- Branch matching

0.58.5. Librarian

Searches knowledge base:

- Related records
- Knowledge base presence
- Context enrichment

0.58.6. MissionControl

Coordinates missions:

- Mission monitoring
- Work effort tracking
- Coordination patterns

0.59. Integration Example

✂ Pantheon: Summoning entities for guidance...

- ✅ Magistrate (Precedent & Proof)
- ✅ Judge (Judgment & Evaluation)
- ✅ TheReasoner (Reasoning Traces)
- ✅ GitHubGod (Repository State)
- ✅ Librarian (Knowledge & Records)

🔊 Judge: Evaluating action safety...
Verdict: PROVEN

Confidence: 0.85

Reasoning: Action is safe for autonomous execution

0.60. Next Steps

Now let's explore Campfire integration.

Auto-Work integrates with Campfire to tell stories around the campfire.

0.61. What is Campfire?

Campfire is a storytelling system that creates narratives about work efforts and actions.

0.62. Integration

After successful execution, Auto-Work:

1. Creates story input from work effort and action
2. Calls Campfire to tell the story
3. Generates story PDF
4. Logs story metadata

0.63. Story Content

Stories include:

- Work effort details (ID, title, status, priority)
- Action taken (type, label, reason)
- Execution instruction
- Context (Empirica, Pantheon, safety gates)

0.64. Example Output

🔥 Campfire: Telling story around the campfire...
Story ID: story-20260119-102530
PDF: _pyrite/campfire/stories/story-20260119-102530.pdf

0.65. Next Steps

Now let's explore D&D campaign integration.

Auto-Work integrates with the D&D campaign system to generate quest PDFs.

0.66. What is D&D Campaign?

The D&D campaign system runs scenarios and generates quest PDFs using Typst templates.

0.67. Integration

After successful execution, Auto-Work:

1. Runs a D&D scenario (encounter, explore, or lore)
2. Generates quest markdown from scenario
3. Creates quest PDF using Typst
4. Saves quest PDF to work effort directory

0.68. Scenario Types

- **[Encounter]**: Combat scenarios
- **[Explore]**: Location discovery
- **[Lore]**: World building

0.69. Example Output

```
× D&D Campaign: Running scenario...  
  Scenario Mode: encounter  
  Quest PDF: _work_efforts/WE-260118-abc1/quest_20260119_102530.pdf
```

0.70. Next Steps

Now let's explore safety mechanisms in detail.

Auto-Work includes comprehensive safety mechanisms to prevent unsafe operations and ensure reliable execution.

0.71. Security Layers

Auto-Work uses multiple layers of security:

1. **[Input Validation]**: Work effort IDs and paths are validated
2. **[Action Whitelisting]**: Only approved action types are allowed
3. **[Command Sanitization]**: Commands are validated and sanitized
4. **[Empirica Gates]**: Epistemic safety gates check operations
5. **[Pantheon Judge]**: Judge evaluates action safety
6. **[Execution Logging]**: All operations are logged

0.72. Layer 1: Input Validation

0.72.1. Work Effort ID Validation

Work effort IDs must match the format: WE-YYMMDD-xxxx

Pattern: `^WE-\d{6}-[a-z0-9]{4}$`

[Example Valid IDs]:

- WE-260118-abc1 
- WE-260117-def2 

[Example Invalid IDs]:

- WE-260118-ABC1  (uppercase not allowed)
- WE-260118-abc  (too short)
- ../WE-260118-abc1  (path injection attempt)

0.72.2. Path Validation

Work effort paths are validated to prevent directory traversal:

- Must be within project root
- Must match work effort ID
- Must exist and be a directory

0.73. Layer 2: Action Whitelisting

Only approved action types are allowed:

Action Type	Description	Allowed	status_transition	Change
work effort status				
add_progress	Add progress note			
review	Review work effort			
review_todos	Review and address TODOs			
fix_issues	Fix issues			
review_changes	Review recent changes			
delete_files	Delete files			
execute_shell	Execute shell commands			

[Example]: If an action type is not in the whitelist, execution is rejected:

 Invalid action type: delete_files
Error: Action type 'delete_files' not in whitelist

0.74. Layer 3: Command Sanitization

Commands are validated before execution:

- Maximum length: 500 characters
- Must not be empty

- Must not contain dangerous patterns
- Must be parameterized (not f-strings)

[Example Safe Command]:

Review and address TODOs in work effort WE-260118-abc1

[Example Rejected Command]:

```
rm -rf / # Too dangerous
```

0.75. Layer 4: Empirica Safety Gates

Empirica provides epistemic safety gates:

0.75.1. Gate Results

| Result | Meaning | Action | |———|———|———| | PROCEED | Safe to execute |  Continue | | HALT | Requires human approval |  Stop | | BRANCH | Needs investigation |  Stop, investigate | | REVISE | Approach needs revision |  Stop, revise |

0.75.2. Gate Check Process

1. Submit operation to Empirica gate
2. Gate evaluates epistemic state
3. Gate returns result (PROCEED/HALT/BRANCH/REVISE)
4. System acts based on result

[Example HALT]:

```
🔊 Empirica Gate: Checking execution safety...
Result: HALT
Reason: Operation requires human approval
```

 Execution halted by safety gate

0.76. Layer 5: Pantheon Judge

The Judge evaluates action safety:

0.76.1. Judge Verdicts

| Verdict | Confidence | Action | |———|———|———| | PROVEN | > 0.9 |  Safe, proceed | | PROVEN | 0.7-0.9 |  Likely safe, proceed | | PROBABLE | > 0.6 |  Probably safe, proceed with caution | | DISPROVEN | > 0.9 |  Unsafe, halt | | DISPROVEN | 0.7-0.9 |  Likely unsafe, halt |

0.76.2. Judge Evaluation

```
🔊 Judge: Evaluating action safety...
Claim: "Action 'review_todos' on work effort WE-260118-abc1 is safe to execute autonomously"
Verdict: PROVEN
Confidence: 0.85
Reasoning: Action is read-only and safe for autonomous execution
```

[Example DISPROVEN]:

```
🔊 Judge: Evaluating action safety...
Verdict: DISPROVEN
Confidence: 0.95
Reasoning: Action involves destructive operations
```

 Execution halted by Judge
Error: Judge DISPROVES action safety (confidence: 0.95)

0.77. Layer 6: Execution Logging

All operations are logged for audit:

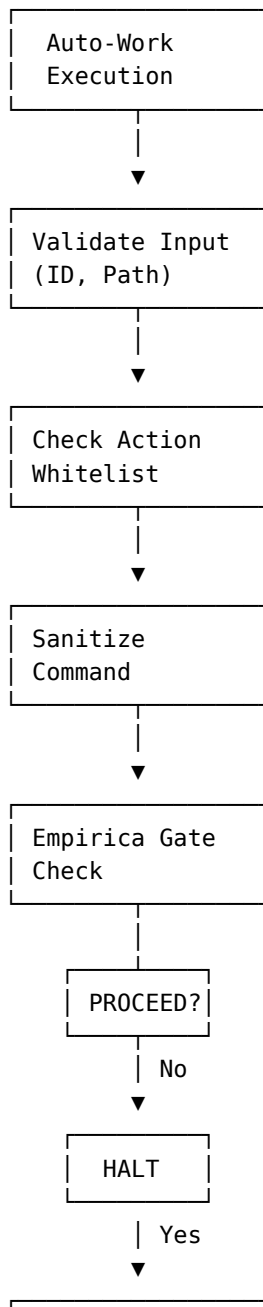
0.77.1. Logged Information

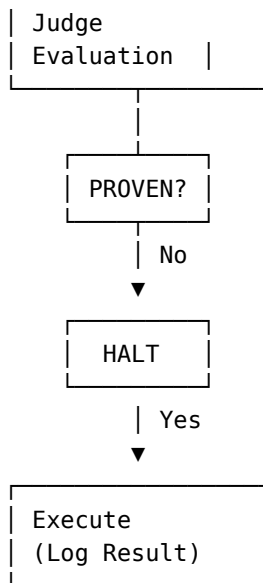
- Work effort ID and title
- Action type and command
- Safety gate results
- Judge verdict
- Execution timestamp
- Success/failure status

0.77.2. Log Locations

- Empirica logs: `_pyrite/empirica/`
- Pantheon traces: `_pantheon/reasoner/traces/`
- Execution logs: Console output

0.78. Safety Flow Diagram





0.79. Security Best Practices

1. **[Always use safety gates]**: Don't bypass Empirica or Judge
2. **[Review gate results]**: Check HALT/BANCH/REVISE results
3. **[Monitor logs]**: Review execution logs regularly
4. **[Validate inputs]**: Ensure work effort IDs are valid
5. **[Use whitelisting]**: Only allow approved action types
6. **[Sanitize commands]**: Validate all commands before execution

0.80. Next Steps

Now that you understand safety mechanisms, let's explore integration with Empirica.

This chapter covers the fundamental usage of Auto-Work, including command syntax and basic examples.

0.81. Command Syntax

The basic command is:

```
/auto-work
```

This command:

1. Analyzes all work efforts
2. Selects the best one
3. Determines the optimal action
4. Prepares execution instruction
5. Outputs JSON for Cursor AI to execute

0.82. Command Options

0.82.1. Dry Run

To see what would be done without executing:

```
/auto-work --dry-run
```

[Screenshot Placeholder: Dry run output]

This is useful for:

- Understanding what Auto-Work would select
- Verifying priority scoring
- Testing without making changes
- Learning how the system works

0.82.2. Verbose Output

For detailed logging:

```
/auto-work --verbose
```

[Screenshot Placeholder: Verbose output showing detailed logs]

Verbose mode shows:

- Detailed priority calculations
- Empirica gate details
- Pantheon entity interactions
- Action analysis details
- Safety gate results

0.82.3. Project Path

To specify a different project path:

```
/auto-work --path /path/to/project
```

0.83. Basic Example

Here's a simple example:

```
$ /auto-work
```

🤔 Thinking about work efforts...

📊 Empirica: Active and monitoring

✗ Pantheon: Summoning entities for guidance...

- ✓ Magistrate (Precedent & Proof)
- ✓ Judge (Judgment & Evaluation)
- ✓ TheReasoner (Reasoning Traces)
- ✓ GitHubGod (Repository State)

📅 Found 15 work effort(s)

✓ 8 actionable work effort(s)

🔮 Selecting best work effort to work on...

✓ Selected: WE-260118-abc1

Title: Implement User Authentication

Status: active

🔍 Analyzing available actions...

✓ Best action: Address TODOs

Reason: Work effort contains TODO items

Command: Review and address TODOs in work effort WE-260118-abc1

🚀 Preparing action...

✓ Work effort and action selected!

=====

AUTO-WORK RESULT (JSON):

=====

```
{
  "selected_work_effort": {
    "id": "WE-260118-abc1",
    "title": "Implement User Authentication",
    "path": "_work_efforts/WE-260118-abc1_implement_user_authentication"
  },
  "action": {
    "type": "review_todos",
    "label": "Address TODOs",
    "command": "Review and address TODOs in work effort WE-260118-abc1",
    "context": {
      "reason": "Work effort contains TODO items",
      "priority": "high"
    }
  },
  "execution_instruction": "Review and address TODOs in work effort WE-260118-abc1"
}
```

=====

🚀 EXECUTING NOW...

=====

EXECUTION INSTRUCTION:

=====

Review and address TODOs in work effort WE-260118-abc1

=====

💡 Cursor AI: Execute the command above now.

[Screenshot Placeholder: Complete basic example output]

0.84. Dry Run Example

Here's what a dry run looks like:

```
$ /auto-work --dry-run
```

🤔 Thinking about work efforts...

📅 Found 15 work effort(s)

✅ 8 actionable work effort(s)

🎯 Selecting best work effort to work on...

✅ Selected: WE-260118-abc1

Title: Implement User Authentication

Status: active

🔍 Analyzing available actions...

✅ Best action: Address TODOs

Reason: Work effort contains TODO items

Command: Review and address TODOs in work effort WE-260118-abc1

🔍 DRY RUN - Would execute:

Review and address TODOs in work effort WE-260118-abc1

[Screenshot Placeholder: Dry run example]

0.85. What Happens After Execution

After Auto-Work outputs the execution instruction:

1. **[Cursor AI reads the JSON]**: Parses the structured output
2. **[AI executes the command]**: Performs the actual work
3. **[Work effort is updated]**: Progress is recorded
4. **[Story is told]**: (Optional) Campfire generates a story
5. **[Quest PDF is generated]**: (Optional) D&D campaign creates a quest

0.86. Common Use Cases

0.86.1. Daily Work Session

Start your day with Auto-Work:

```
$ /auto-work
```

This automatically selects and starts work on the highest priority item.

0.86.2. Before Committing

Use dry run to see what would be worked on:

```
$ /auto-work --dry-run
```

0.86.3. Debugging Priority

Use verbose mode to understand priority scoring:

```
$ /auto-work --verbose
```

0.87. Next Steps

Now that you understand basic usage, let's explore command options in detail.

This chapter details all command-line options for Auto-Work.

0.88. Basic Command

```
/auto-work
```

Executes Auto-Work with default settings.

0.89. Options

0.89.1. --dry-run

Preview what would be done without executing:

```
/auto-work --dry-run
```

0.89.2. --verbose

Enable detailed logging:

```
/auto-work --verbose
```

0.89.3. --path

Specify project path:

```
/auto-work --path /path/to/project
```

0.90. Option Combinations

0.90.1. Dry Run with Verbose

```
/auto-work --dry-run --verbose
```

0.90.2. Custom Path with Verbose

```
/auto-work --path /path/to/project --verbose
```

0.91. Next Steps

Now let's explore troubleshooting.

This chapter provides a detailed walkthrough of Auto-Work execution from start to finish, with example outputs at each step.

0.92. Prerequisites Check

Before starting, ensure:

- Work efforts exist in `_work_efforts/` directory
- At least one work effort is not completed
- (Optional) Empirica is initialized
- (Optional) Pantheon entities are available

0.93. Step 1: Command Execution

Run the Auto-Work command:

```
/auto-work
```

[Screenshot Placeholder: Terminal showing command execution]

🤔 Thinking about work efforts...

0.94. Step 2: System Initialization

The system initializes supporting systems:

0.94.1. Empirica Initialization

🐼 Empirica: Active and monitoring

If Empirica is not initialized:

⚠ Empirica: Not initialized (continuing without epistemic tracking)

0.94.2. Pantheon Initialization

🔮 Pantheon: Summoning entities for guidance...

- ✅ Magistrate (Precedent & Proof)
- ✅ Judge (Judgment & Evaluation)
- ✅ TheReasoner (Reasoning Traces)
- ✅ GitHubGod (Repository State)
- ✅ Fae (Quests & Creativity)
- ✅ MissionControl (Coordination)
- ✅ Librarian (Knowledge & Records)

If Pantheon is unavailable:

⚠ Pantheon: Not available (continuing without Pantheon guidance)

0.94.3. Campfire Initialization

🔥 Campfire: Ready for storytelling

0.94.4. D&D Campaign Initialization

✖ D&D Campaign: Initializing realm and quest system...

- ✅ Scenario Realm initialized
- ✅ Scenario Orchestrator ready
- ✅ Quest PDF Generator ready (Typst available)

[Screenshot Placeholder: System initialization output]

0.95. Step 3: Work Effort Collection

The system gathers all work efforts:

 Found 15 work effort(s)

 8 actionable work effort(s)

[Screenshot Placeholder: Work effort collection output]

0.96. Step 4: Priority Scoring

For each actionable work effort, the system calculates priority scores:

 Selecting best work effort to work on...

Calculating priorities...

WE-260118-abc1: 230.0 points (active, HIGH, TODO, FIXME, 2 commits)

WE-260118-def2: 150.0 points (active, MEDIUM, TODO)

WE-260117-ghi3: 100.0 points (paused, HIGH)

...

[Screenshot Placeholder: Priority scoring output]

0.97. Step 5: Work Effort Selection

The highest scoring work effort is selected:

 Selected: WE-260118-abc1

Title: Implement User Authentication

Status: active

Priority Score: 230.0

[Screenshot Placeholder: Selected work effort details]

0.98. Step 6: Action Analysis

The system analyzes available actions for the selected work effort:

 Analyzing available actions...

Available actions:

1. Address TODOs (HIGH priority)
Reason: Work effort contains TODO items
2. Review changes (MEDIUM priority)
Reason: Recent changes detected
3. Add progress note (LOW priority)
Reason: Work effort is active

[Screenshot Placeholder: Action analysis output]

0.99. Step 7: Action Selection

The highest priority action is selected:

 Best action: Address TODOs

Reason: Work effort contains TODO items

Command: Review and address TODOs in work effort WE-260118-abc1

[Screenshot Placeholder: Selected action details]

0.100. Step 8: Safety Gates

The system runs safety checks:

0.100.1. Empirica Gate

 Empirica Gate: Checking execution safety...
Result: PROCEED

If gate returns HALT:

 Empirica Gate: Checking execution safety...
Result: HALT
Reason: Operation requires human approval

✖ Execution halted by safety gate

0.100.2. Pantheon Judge

 Judge: Evaluating action safety...
Verdict: PROVEN (confidence: 0.85)
Reasoning: Action is safe to execute autonomously

[Screenshot Placeholder: Safety gate results]

0.101. Step 9: Execution Preparation

The system prepares the execution instruction:

 Preparing action...

✔ Work effort and action selected!

[Screenshot Placeholder: Execution preparation output]

0.102. Step 10: JSON Output

The system outputs structured JSON for Cursor AI:

```
=====
AUTO-WORK RESULT (JSON):
=====
{
  "selected_work_effort": {
    "id": "WE-260118-abc1",
    "title": "Implement User Authentication",
    "path": "_work_efforts/WE-260118-abc1_implement_user_authentication"
  },
  "action": {
    "type": "review_todos",
    "label": "Address TODOs",
    "command": "Review and address TODOs in work effort WE-260118-abc1",
    "context": {
      "reason": "Work effort contains TODO items",
      "priority": "high"
    }
  },
  "execution_instruction": "Review and address TODOs in work effort WE-260118-abc1"
}
=====
```

[Screenshot Placeholder: JSON output]

0.103. Step 11: Execution Instruction

The system displays the execution instruction:

 EXECUTING NOW...

=====

EXECUTION INSTRUCTION:

=====

Review and address TODOs in work effort WE-260118-abc1

=====

💡 Cursor AI: Execute the command above now.

[Screenshot Placeholder: Execution instruction]

0.104. Step 12: AI Execution

Cursor AI executes the instruction, performing the actual work.

[Screenshot Placeholder: AI execution in progress]

0.105. Step 13: Storytelling (Optional)

If Campfire is available, a story is told:

🔥 Campfire: Telling story around the campfire...
Story ID: story-20260119-102530
PDF: _pyrite/campfire/stories/story-20260119-102530.pdf

[Screenshot Placeholder: Story generation output]

0.106. Step 14: Quest Generation (Optional)

If D&D campaign is available, a quest PDF is generated:

× D&D Campaign: Running scenario...
Scenario Mode: encounter
Quest PDF: _work_efforts/WE-260118-abc1/quest_20260119_102530.pdf

[Screenshot Placeholder: Quest PDF generation]

0.107. Complete Example Output

Here's a complete example of Auto-Work execution:

🤔 Thinking about work efforts...

📊 Empirica: Active and monitoring

⚔️ Pantheon: Summoning entities for guidance...

- ✅ Magistrate (Precedent & Proof)
- ✅ Judge (Judgment & Evaluation)
- ✅ TheReasoner (Reasoning Traces)
- ✅ GitHubGod (Repository State)
- ✅ Librarian (Knowledge & Records)

🔥 Campfire: Ready for storytelling

× D&D Campaign: Initializing realm and quest system...

- ✅ Scenario Realm initialized
- ✅ Scenario Orchestrator ready
- ✅ Quest PDF Generator ready (Typst available)

📁 Found 15 work effort(s)

✅ 8 actionable work effort(s)

🔍 Selecting best work effort to work on...

✅ Selected: WE-260118-abc1
Title: Implement User Authentication
Status: active

🔍 Analyzing available actions...

✅ Best action: Address TODOs
Reason: Work effort contains TODO items
Command: Review and address TODOs in work effort WE-260118-abc1

🚀 Preparing action...

✅ Work effort and action selected!

=====

AUTO-WORK RESULT (JSON):

=====

```
{
  "selected_work_effort": {
    "id": "WE-260118-abc1",
    "title": "Implement User Authentication",
    "path": "_work_efforts/WE-260118-abc1_implement_user_authentication"
  },
  "action": {
    "type": "review_todos",
    "label": "Address TODOs",
    "command": "Review and address TODOs in work effort WE-260118-abc1",
    "context": {
      "reason": "Work effort contains TODO items",
      "priority": "high"
    }
  },
  "execution_instruction": "Review and address TODOs in work effort WE-260118-abc1"
}
```

=====

🚀 EXECUTING NOW...

=====

EXECUTION INSTRUCTION:

=====

Review and address TODOs in work effort WE-260118-abc1

=====

💡 Cursor AI: Execute the command above now.

🔥 Campfire: Story told around the campfire
Story ID: story-20260119-102530
PDF: _pyrite/campfire/stories/story-20260119-102530.pdf

× D&D Campaign: Quest PDF generated
Quest: Quest: Implement User Authentication
PDF: _work_efforts/WE-260118-abc1/quest_20260119_102530.pdf

[Screenshot Placeholder: Complete terminal output]

0.108. Next Steps

Now that you've seen the complete walkthrough, let's explore usage examples in different scenarios.

This chapter provides real-world examples of Auto-Work in different scenarios.

0.109. Example 1: Simple Execution

[Scenario]: You have multiple work efforts and want Auto-Work to select and execute the best one.

[Command]:

```
/auto-work
```

[Output]:

🤔 Thinking about work efforts...

📄 Found 5 work effort(s)

✅ 3 actionable work effort(s)

🎯 Selecting best work effort to work on...

✅ Selected: WE-260118-abc1

Title: Fix Critical Bug in Authentication

Status: active

🔍 Analyzing available actions...

✅ Best action: Fix Issues

Reason: Work effort contains bug references

Command: Review and fix issues in work effort WE-260118-abc1

🚀 Preparing action...

✅ Work effort and action selected!

=====

AUTO-WORK RESULT (JSON):

=====

```
{
  "selected_work_effort": {
    "id": "WE-260118-abc1",
    "title": "Fix Critical Bug in Authentication",
    "path": "_work_efforts/WE-260118-abc1_fix_critical_bug"
  },
  "action": {
    "type": "fix_issues",
    "label": "Fix Issues",
    "command": "Review and fix issues in work effort WE-260118-abc1",
    "context": {
      "reason": "Work effort contains bug references",
      "priority": "high"
    }
  },
  "execution_instruction": "Review and fix issues in work effort WE-260118-abc1"
}
```

=====

[Screenshot Placeholder: Example 1 output]

0.110. Example 2: Dry Run Before Execution

[Scenario]: You want to see what Auto-Work would select before actually executing.

[Command]:

```
/auto-work --dry-run
```

[Output]:

🤔 Thinking about work efforts...

📅 Found 5 work effort(s)

✅ 3 actionable work effort(s)

🔮 Selecting best work effort to work on...

✅ Selected: WE-260118-abc1

Title: Fix Critical Bug in Authentication

Status: active

🔍 Analyzing available actions...

✅ Best action: Fix Issues

Reason: Work effort contains bug references

Command: Review and fix issues in work effort WE-260118-abc1

🔍 DRY RUN - Would execute:

Review and fix issues in work effort WE-260118-abc1

[Screenshot Placeholder: Example 2 dry run output]

0.111. Example 3: With Empirica and Pantheon

[Scenario]: Auto-Work with full integration (Empirica, Pantheon, Campfire, D&D).

[Command]:

```
/auto-work
```

[Output]:

🤔 Thinking about work efforts...

🧙 Empirica: Active and monitoring

🔮 Pantheon: Summoning entities for guidance...

✅ Magistrate (Precedent & Proof)

✅ Judge (Judgment & Evaluation)

✅ TheReasoner (Reasoning Traces)

✅ GitHubGod (Repository State)

✅ Librarian (Knowledge & Records)

🔥 Campfire: Ready for storytelling

✖ D&D Campaign: Initializing realm and quest system...

✅ Scenario Realm initialized

✅ Scenario Orchestrator ready

✅ Quest PDF Generator ready (Typst available)

📅 Found 15 work effort(s)

✅ 8 actionable work effort(s)

🌀 Selecting best work effort to work on...

✅ Selected: WE-260118-abc1
Title: Implement User Authentication
Status: active
Priority Score: 230.0

🔍 Analyzing available actions...

✅ Best action: Address TODOs
Reason: Work effort contains TODO items
Command: Review and address TODOs in work effort WE-260118-abc1

🚀 Preparing action...

🔬 Empirica Gate: PROCEED
⚖ Judge: PROVEN (confidence: 0.85)

✅ Work effort and action selected!

=====

AUTO-WORK RESULT (JSON):

=====

```
{
  "selected_work_effort": {
    "id": "WE-260118-abc1",
    "title": "Implement User Authentication",
    "path": "_work_efforts/WE-260118-abc1_implement_user_authentication"
  },
  "action": {
    "type": "review_todos",
    "label": "Address TODOs",
    "command": "Review and address TODOs in work effort WE-260118-abc1",
    "context": {
      "reason": "Work effort contains TODO items",
      "priority": "high"
    }
  },
  "execution_instruction": "Review and address TODOs in work effort WE-260118-abc1"
}
```

=====

🔥 Campfire: Story told around the campfire
Story ID: story-20260119-102530
PDF: _pyrite/campfire/stories/story-20260119-102530.pdf

× D&D Campaign: Quest PDF generated
Quest: Quest: Implement User Authentication
PDF: _work_efforts/WE-260118-abc1/quest_20260119_102530.pdf

[Screenshot Placeholder: Example 3 full integration output]

0.112. Example 4: Safety Gate Halt

[Scenario]: Empirica safety gate halts execution due to high risk.

[Command]:

/auto-work

[Output]:

🤔 Thinking about work efforts...

🔬 Empirica: Active and monitoring

📅 Found 5 work effort(s)

✅ 3 actionable work effort(s)

🔧 Selecting best work effort to work on...

✅ Selected: WE-260118-xyz9

Title: Delete Production Database

Status: active

🔍 Analyzing available actions...

✅ Best action: Execute Deletion

Reason: Work effort is ready for execution

Command: Execute deletion in work effort WE-260118-xyz9

🚀 Preparing action...

🔬 Empirica Gate: HALT

Reason: Operation requires human approval

❌ Execution halted by safety gate

Error: Empirica gate: Operation requires human approval

Gate Result: HALT

[Screenshot Placeholder: Example 4 safety halt]

0.113. Example 5: No Actionable Work Efforts

[Scenario]: All work efforts are completed.

[Command]:

/auto-work

[Output]:

🤔 Thinking about work efforts...

📅 Found 5 work effort(s)

✅ 0 actionable work effort(s)

❌ No actionable work efforts found (all completed).

[Screenshot Placeholder: Example 5 no actionable work efforts]

0.114. Example 6: Verbose Mode

[Scenario]: You want detailed logging to understand priority scoring.

[Command]:

/auto-work --verbose

[Output] (excerpt):

🤔 Thinking about work efforts...

📄 Found 5 work effort(s)

✅ 3 actionable work effort(s)

🔍 Selecting best work effort to work on...

Calculating priority for WE-260118-abc1:

Status: active → 100.0 points
Priority: HIGH → +30.0 points
Content: TODO found → +20.0 points
Content: FIXME found → +25.0 points
Git activity: 2 commits → +10.0 points
Empirica: PROCEED → +10.0 points
Judge: PROVEN (0.85) → +15.0 points
Total: 230.0 points

Calculating priority for WE-260118-def2:

Status: active → 100.0 points
Priority: MEDIUM → +15.0 points
Content: TODO found → +20.0 points
Git activity: 1 commit → +5.0 points
Total: 140.0 points

✅ Selected: WE-260118-abc1 (230.0 points)

[Screenshot Placeholder: Example 6 verbose output]

0.115. Example 7: Multiple Work Efforts with Different Priorities

[Scenario]: You have work efforts with varying priorities and statuses.

[Work Efforts]:

- WE-260118-abc1: active, CRITICAL, TODO, FIXME
- WE-260118-def2: active, HIGH, TODO
- WE-260117-ghi3: paused, HIGH
- WE-260116-jkl4: open, MEDIUM

[Command]:

/auto-work --dry-run

[Expected Selection]: WE-260118-abc1 (highest score due to CRITICAL priority and multiple content indicators)

[Screenshot Placeholder: Example 7 priority comparison]

0.116. Best Practices from Examples

1. [Use dry run first]: Always test with --dry-run before actual execution
2. [Check safety gates]: Review Empirica and Judge results
3. [Monitor verbose output]: Use --verbose to understand decisions
4. [Review JSON output]: Check the structured output for accuracy
5. [Verify execution]: Confirm the AI executed the intended action

0.117. Next Steps

Now that you've seen various examples, let's explore troubleshooting common issues.

This chapter covers common issues and solutions.

0.118. No Work Efforts Found

[Problem]: ✖ No work efforts found.

[Solution]:

- Check that _work_efforts/ directory exists
- Verify work effort directories follow WE-YYMMDD-xxxx format
- Ensure work efforts are in the correct location

0.119. No Actionable Work Efforts

[Problem]: ✖ No actionable work efforts found (all completed).

[Solution]:

- Create new work efforts
- Reopen paused work efforts
- Change status from completed to active or paused

0.120. Action Not Available

[Problem]: ✖ No actions available for this work effort.

[Solution]:

- Check work effort has index file
- Verify work effort structure
- Ensure work effort is not empty

0.121. Safety Gate Halt

[Problem]: ✖ Execution halted by safety gate

[Solution]:

- Review Empirica gate reason
- Check Judge verdict
- Manually approve if safe
- Revise approach if needed

0.122. Integration Unavailable

[Problem]: ⚠ Empirica: Not initialized

[Solution]:

- Initialize Empirica (optional)
- System continues without it
- Check initialization logs

0.123. Next Steps

Now let's explore customization options.

This chapter covers customization options for Auto-Work.

0.124. Priority Scoring Customization

You can adjust priority scoring by modifying:

- Status weights
- Priority level weights
- Content indicator weights
- Activity weights
- Empirica adjustments
- Pantheon adjustments

0.125. Action Types

Add custom action types by:

1. Extending action analyzer
2. Adding to whitelist
3. Implementing action handler

0.126. Integration Configuration

Configure integrations:

- Empirica: Initialize in project
- Pantheon: Configure entities
- Campfire: Set up storytelling
- D&D: Configure campaign system

0.127. Next Steps

Now let's explore best practices.

This chapter covers best practices for using Auto-Work.

0.128. Work Effort Management

- Keep work efforts organized
- Use clear, descriptive titles
- Set appropriate priorities
- Update status regularly
- Add TODOs and FIXMEs for tracking

0.129. Safety

- Always review safety gate results
- Use dry run before execution
- Monitor execution logs
- Verify work effort IDs
- Check action types

0.130. Integration

- Initialize Empirica for epistemic tracking
- Configure Pantheon for decision support
- Set up Campfire for storytelling
- Enable D&D campaign for quest generation

0.131. Execution

- Use dry run to preview
- Review JSON output
- Verify execution results
- Monitor logs
- Update work efforts after execution

0.132. Next Steps

Now let's explore future enhancements.

This chapter covers planned future enhancements.

0.133. Planned Features

- Multi-work-effort execution (batch mode)
- Learning from execution results
- Adaptive priority scoring
- Execution history and analytics
- Custom action types
- Work effort templates
- Integration with more systems

0.134. Enhancement Ideas

- Machine learning for priority prediction
- Collaborative filtering for work effort selection
- Real-time priority updates
- Work effort dependencies
- Execution scheduling
- Resource monitoring

0.135. Next Steps

Now let's conclude the guide.

This guide has covered the WAFT Auto-Work feature comprehensively, from basic usage to advanced integration.

0.136. Key Takeaways

1. **[Auto-Work is intelligent]**: Uses sophisticated algorithms for prioritization
2. **[Auto-Work is safe]**: Multiple layers of security ensure safe operation
3. **[Auto-Work is integrated]**: Works seamlessly with Empirica, Pantheon, Campfire, and D&D
4. **[Auto-Work is flexible]**: Supports dry run, verbose mode, and customization

0.137. Next Steps

- Try Auto-Work with `/auto-work --dry-run`
- Review priority scoring with `--verbose`
- Configure integrations (Empirica, Pantheon, Campfire, D&D)
- Explore customization options
- Monitor execution logs

0.138. Resources

- WAFT Documentation: `docs/`
- Auto-Work Script: `scripts/auto_work.py`
- Command Reference: `.cursor/commands/auto-work.md`
- Work Effort System: `_work_efforts/`

0.139. Feedback

For feedback or questions:

- GitHub Issues: <https://github.com/ctavolazzi/waft/issues>
- Documentation: `docs/`
- Work Efforts: Create a work effort for enhancements

Thank you for using WAFT Auto-Work!

